Deep Learning in Computer Vision· Lecture 4 of 9

Convolutional Neural Networks (CNNs)

Building blocks: convolution, ReLU, batch normalization, pooling, fully connected layers

Recap and Goals

- Last week: neurons, fully connected nets, training, evaluation
- Problem: **150M weights** for one layer on a 224×224 image; no spatial structure
- Today: the **CNN**, designed for images
- By the end you can trace an image through a CNN and **compute output shapes and parameter counts**

CNN

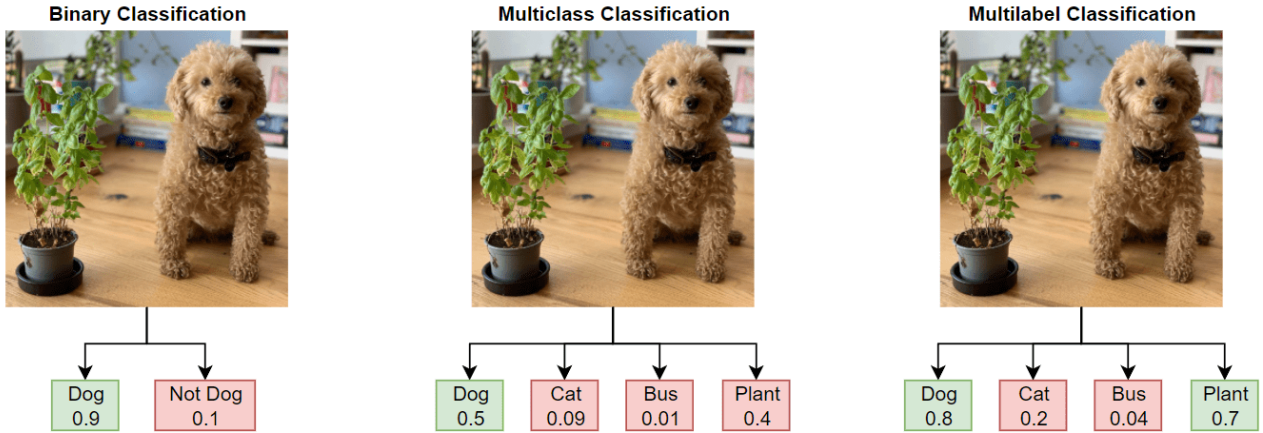
- A Convolutional Neural Network (CNN) is a type of neural network that is especially good at analyzing visual data like images.
- CNN works by automatically detecting patterns, like edges, shapes, and textures, to recognize objects within the image.
- Think of a CNN like a series of filters on an image.
- Imagine looking at a photo through different magnifying glasses.
  - One glass helps you see edges, another shows you textures, and another might help you see colors.
  - By stacking these views, a CNN learns to recognize complex patterns, like faces or animals, based on simpler shapes and textures.

CNN

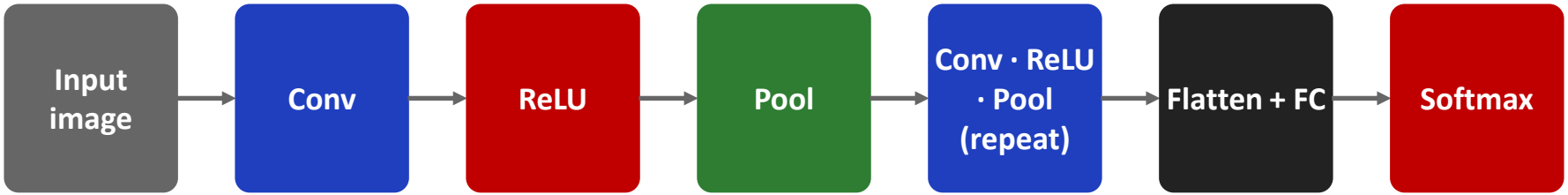
- Convolutional filters, also called kernels are designed to detect specific patterns or features in the input data.
  - For example, in image processing, filters might be designed to detect edges, corners, or textures.
- In deep learning, the weights of these filters are learned automatically through training on large datasets.
  - The **output matrix** is called **feature map (or Tensor)**.
    - Feature maps are not directly interpretable by humans, but they serve as a way for the network to capture and encode the presence of specific features in the input data.
  - As the network learns from data during training, it adapts its filters to recognize features that are relevant to the task at hand, such as classifying objects in images.

CNN Applications: Image Classification

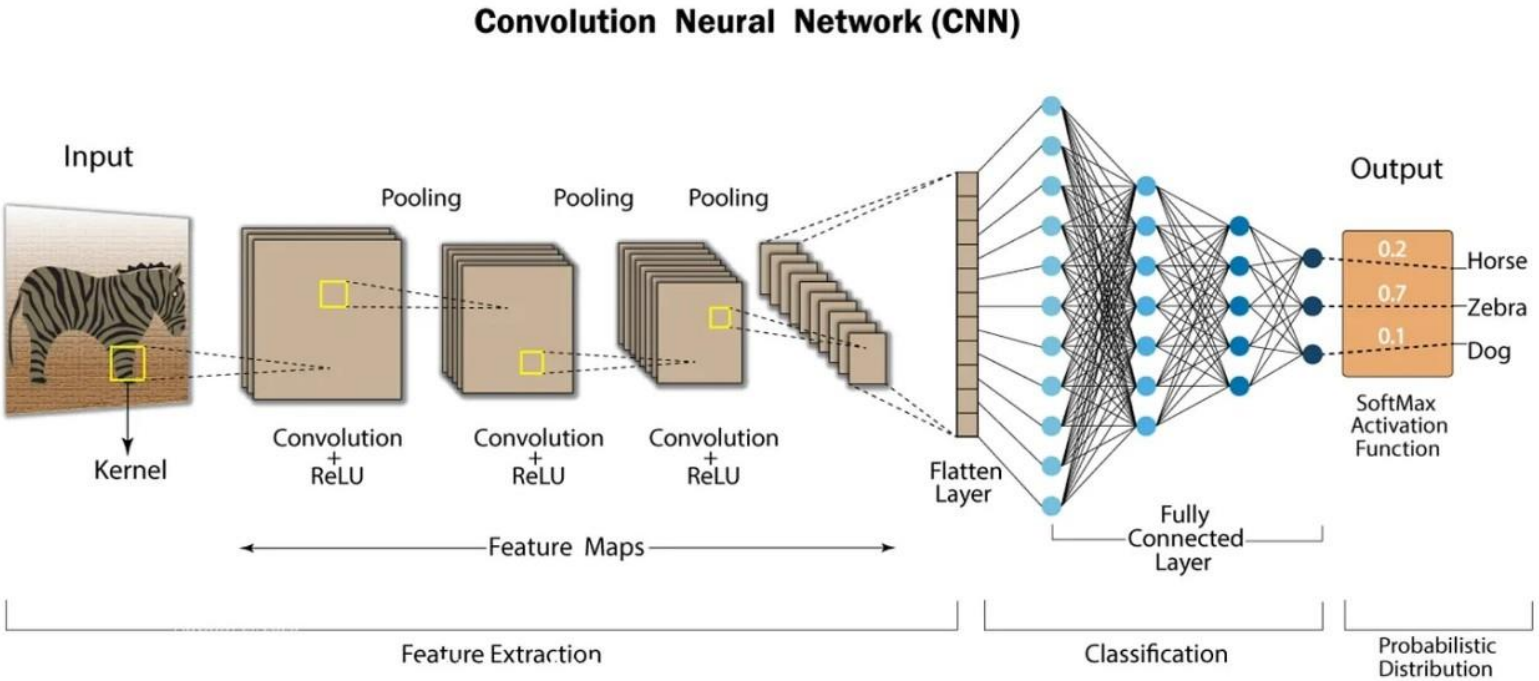
- Assign an image to one or more **predefined classes**
  - **Binary**: tumor / no tumor
  - **Multiclass**: exactly one of K classes (softmax)
  - **Multilabel**: several labels at once (sigmoid per label)
- Output of a CNN = class scores for the whole image



The CNN Pipeline



- **Feature extractor** (conv + ReLU + pool blocks): finds edges, textures, parts
- **Classifier** (fully connected + softmax): decides the class from those features
- Everything is trained **end to end** with backprop (Lecture 2)


Convolution Neural Network (CNN)
Input
Pooling
Pooling
Pooling
Output
0.2
Horse
0.7
Zebra
0.1
Dog
SoftMax
Activation
Function
Kernel
Convolution
+
ReLU
Convolution
+
ReLU
Convolution
+
ReLU
Flatten
Layer
Fully
Connected
Layer
Feature Maps
Feature Extraction
Classification
Probabilistic
Distribution

Two Ideas Behind Convolution

Local connectivity

Each neuron looks at a **small patch** (e.g. 3×3), not the whole image. Nearby pixels belong together.

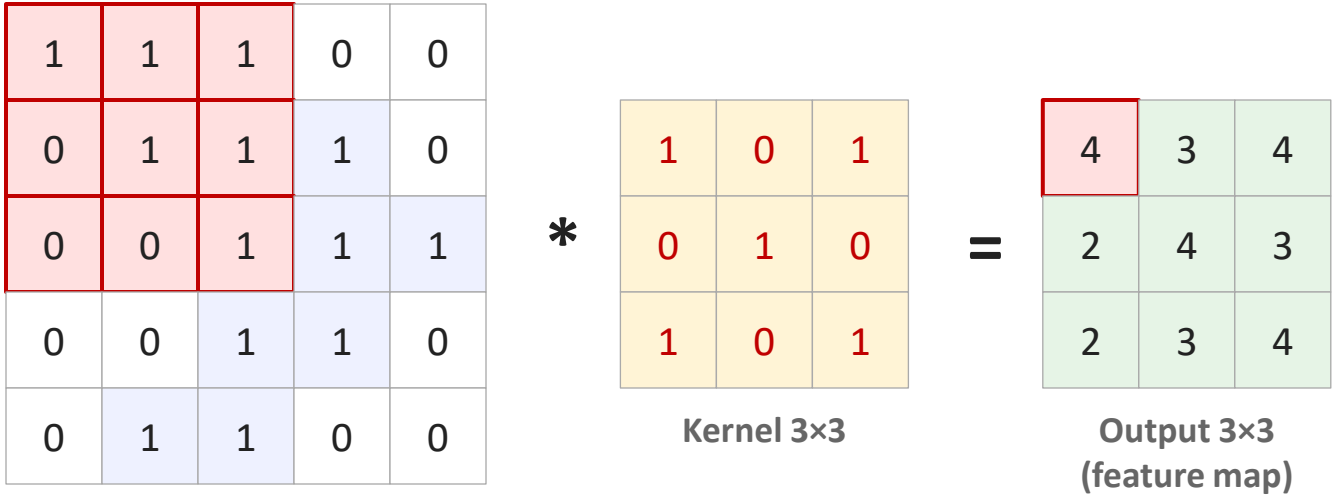
Weight sharing

The **same kernel** slides over the whole image. E.g., one cat detector works anywhere.

Result

Far fewer parameters, and a feature found in one place is found in **every** place (translation equivariance).

The Convolution Operation



- Slide the kernel; **multiply elementwise and sum**
- Highlighted: 1·1+1·0+1·1+0·0+1·1 +1·0+0·1+0·0+1·1 = **4**

Output is smaller: 5 – 3 + 1 = 3 per side (no padding, stride 1)

- **see next slide for how this is computed**

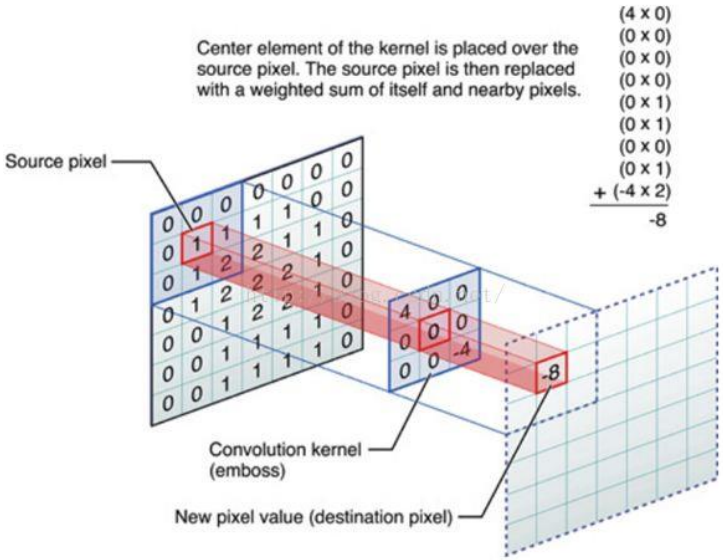
Stride and Padding

- **Stride S**: how far the kernel jumps (S = 2 halves the size)
- **Padding P**: add a border of zeros; "same" padding keeps size
- Output size per side:

$$O = \text{floor}((W - K + 2P) / S) + 1$$

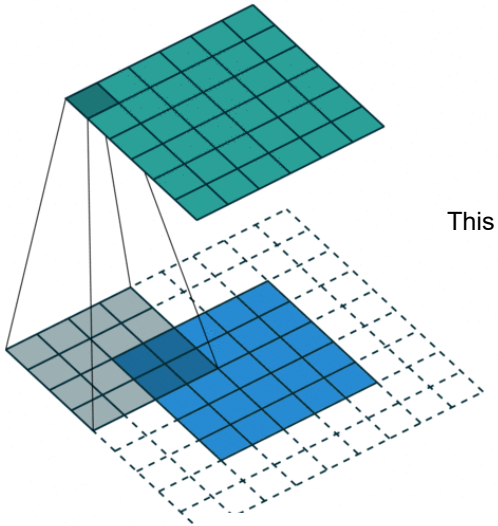
Input W	Kernel K	Pad P	Stride S	Output O
32	5	0	1	28
224	3	1	1	224 (same)
224	7	3	2	112
28	3	0	1	26

Convolutional kernel



Padding on the input volume with zeros in such way that the conv layer does not alter the spatial dimensions of the input

Example: Padding =2 and Stride =1



This is an animated image

Kernels Detect Features

1	0	-1
1	0	-1
1	0	-1

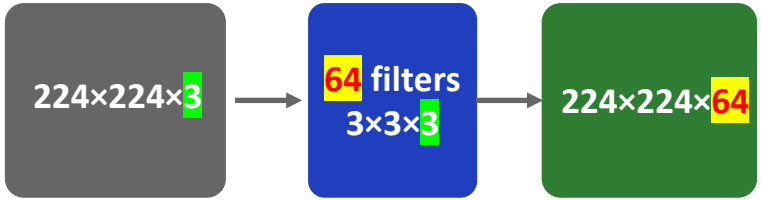
Vertical-edge kernel

1	1	1
0	0	0
-1	-1	-1

Horizontal-edge kernel

- Classic filters were **hand-designed** (Sobel, blur, sharpen)
- In a CNN the kernel values are **weights learned by backprop**
- First layers learn edges and colors; deeper layers learn **textures, parts, objects**
- Each kernel produces one **feature map** showing where that pattern occurs

Color Images and Many Filters

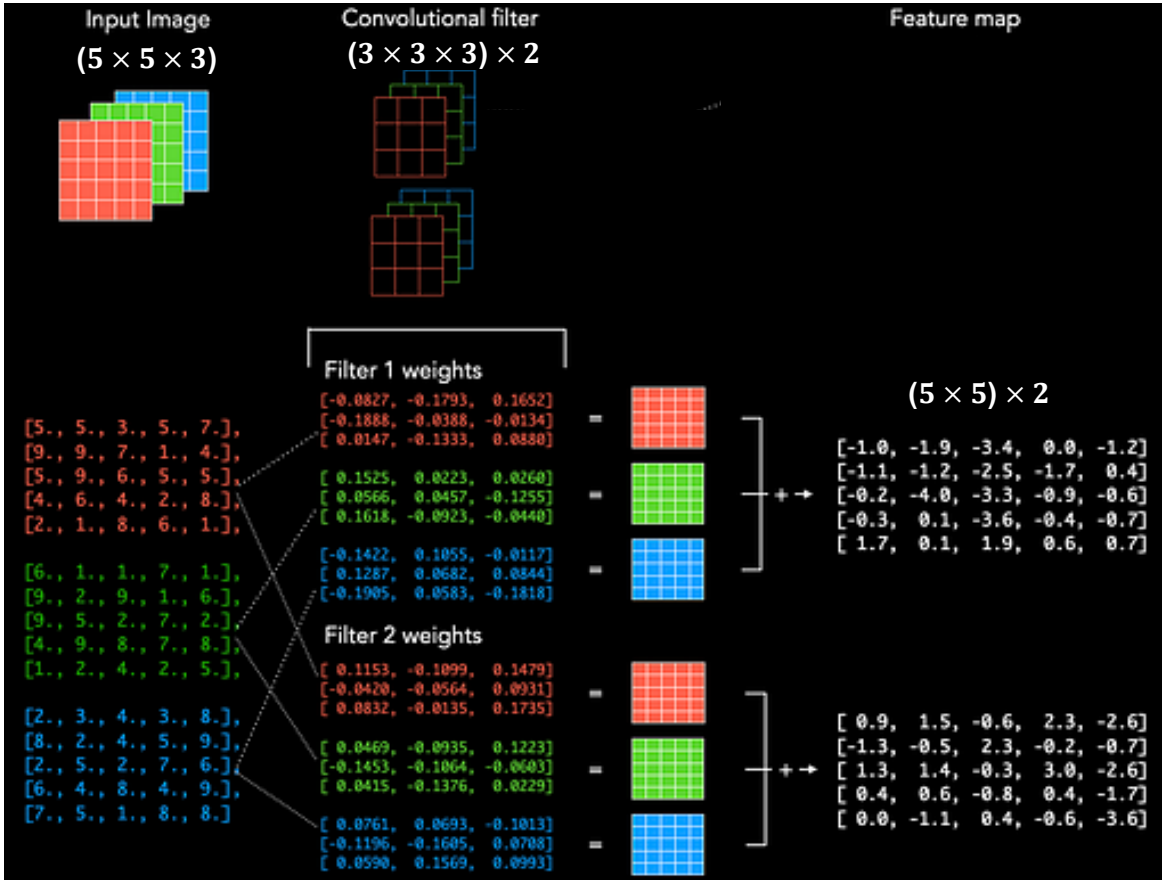


(same padding, stride 1)

- An RGB image is a **H × W × 3** volume
- A kernel is **K × K × C_in** (e.g. 3×3×3):
  - **C_in**: input channels
  - **it spans all channels and gives one number per position**
- Use **C_out filters** → output has **C_out feature maps** (channels)
  - Parameters: **K × K × C_in × C_out + C_out (Bias Parameters)**
  - Example: 3×3×3, 64 filters → 3 × 3 × 3 × 64 + 64 = **1,792** parameters

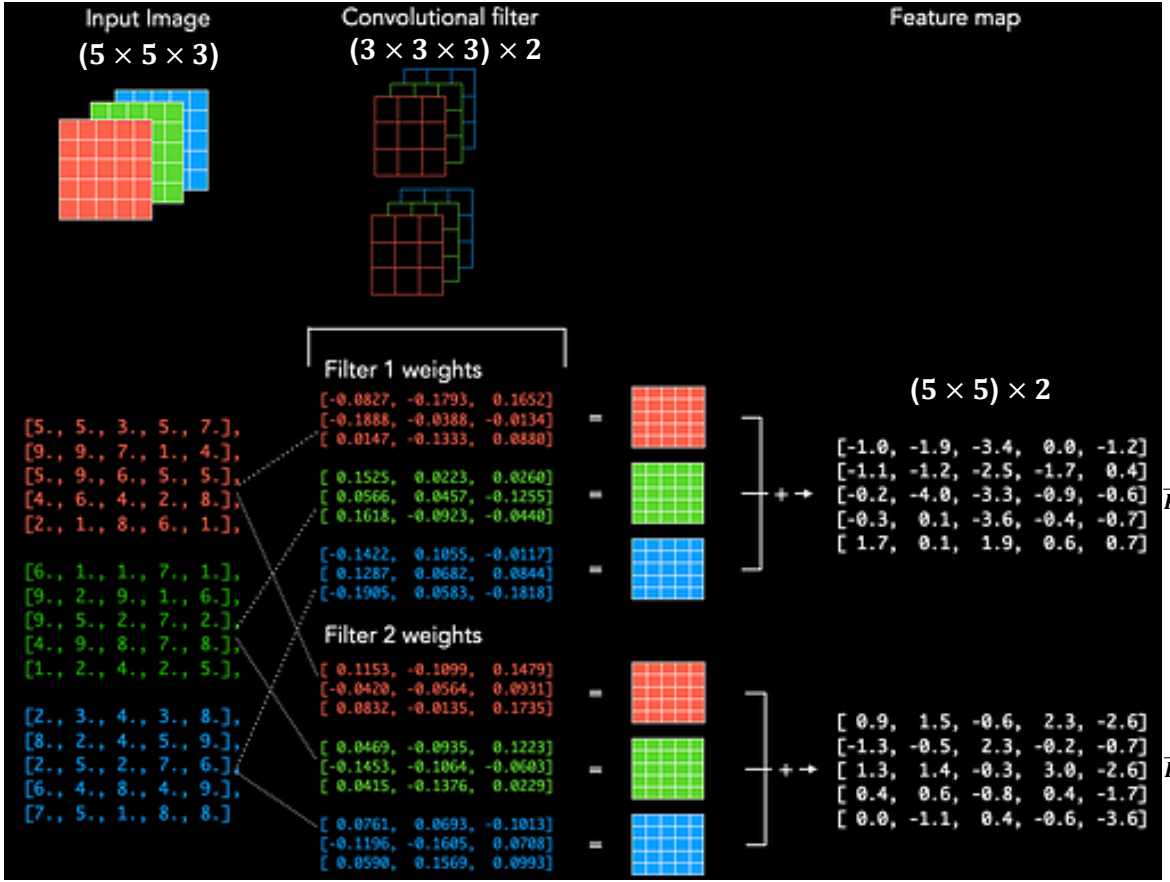
Convolutional operation on colored RGB images

Example with random numbers (Padding = 1, Stride=1)



Convolutional operation on colored RGB images

Example with random numbers



$$\overrightarrow{ReLU} = \begin{bmatrix} 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.4 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.1 & 0.0 & 0.0 & 0.0 \\ 1.7 & 0.1 & 1.9 & 0.6 & 0.7 \end{bmatrix}$$

$$\overrightarrow{ReLU} = \begin{bmatrix} 0.9 & 1.5 & 0.0 & 2.3 & 0.0 \\ 0.0 & 0.0 & 2.3 & 0.0 & 0.0 \\ 1.3 & 1.4 & 0.0 & 3.0 & 0.0 \\ 0.4 & 0.6 & 0.0 & 0.4 & 0.0 \\ 0.0 & 0.0 & 0.4 & 0.0 & 0.0 \end{bmatrix}$$

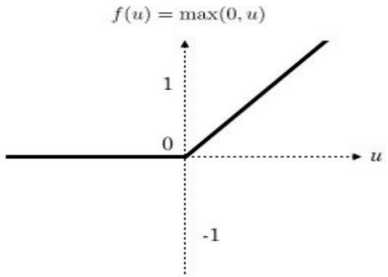
ReLU: Adding Non-linearity

ReLU(x) = max(0, x)

3	-2	1
-1	4	-5
2	0	-3

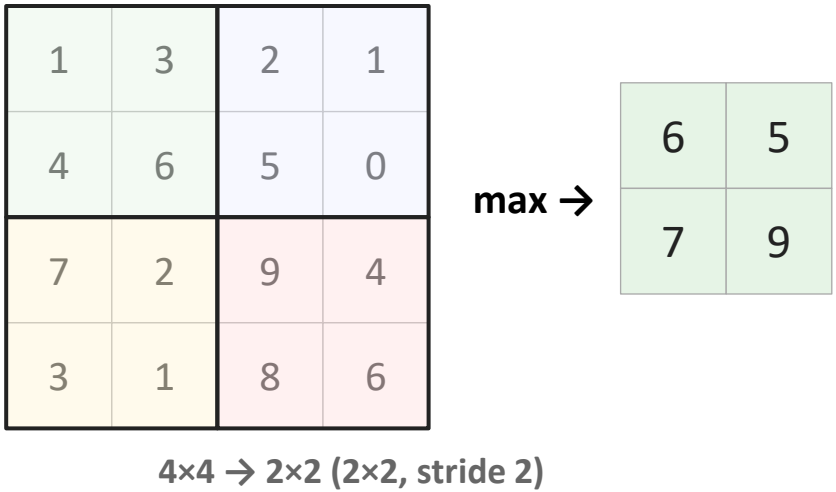
→

3	0	1
0	4	0
2	0	0



- Applied **after every convolution**, elementwise
- Negative values → 0; keeps positive evidence
- Without it, stacked convs collapse to **one linear filter**
- Cheap, and gradient is 1 for x > 0: helps against **vanishing gradients**

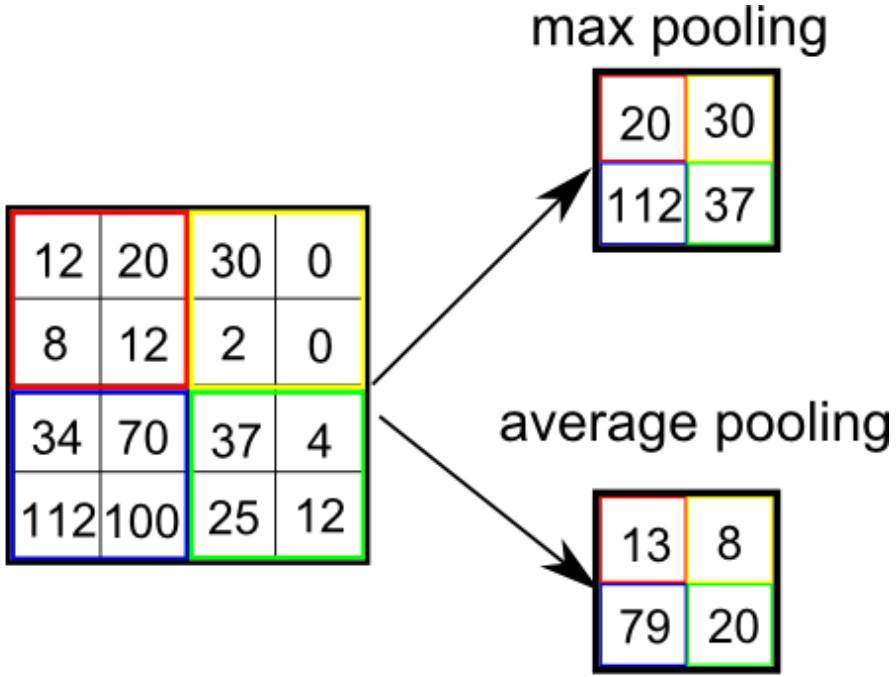
Pooling: Downsampling



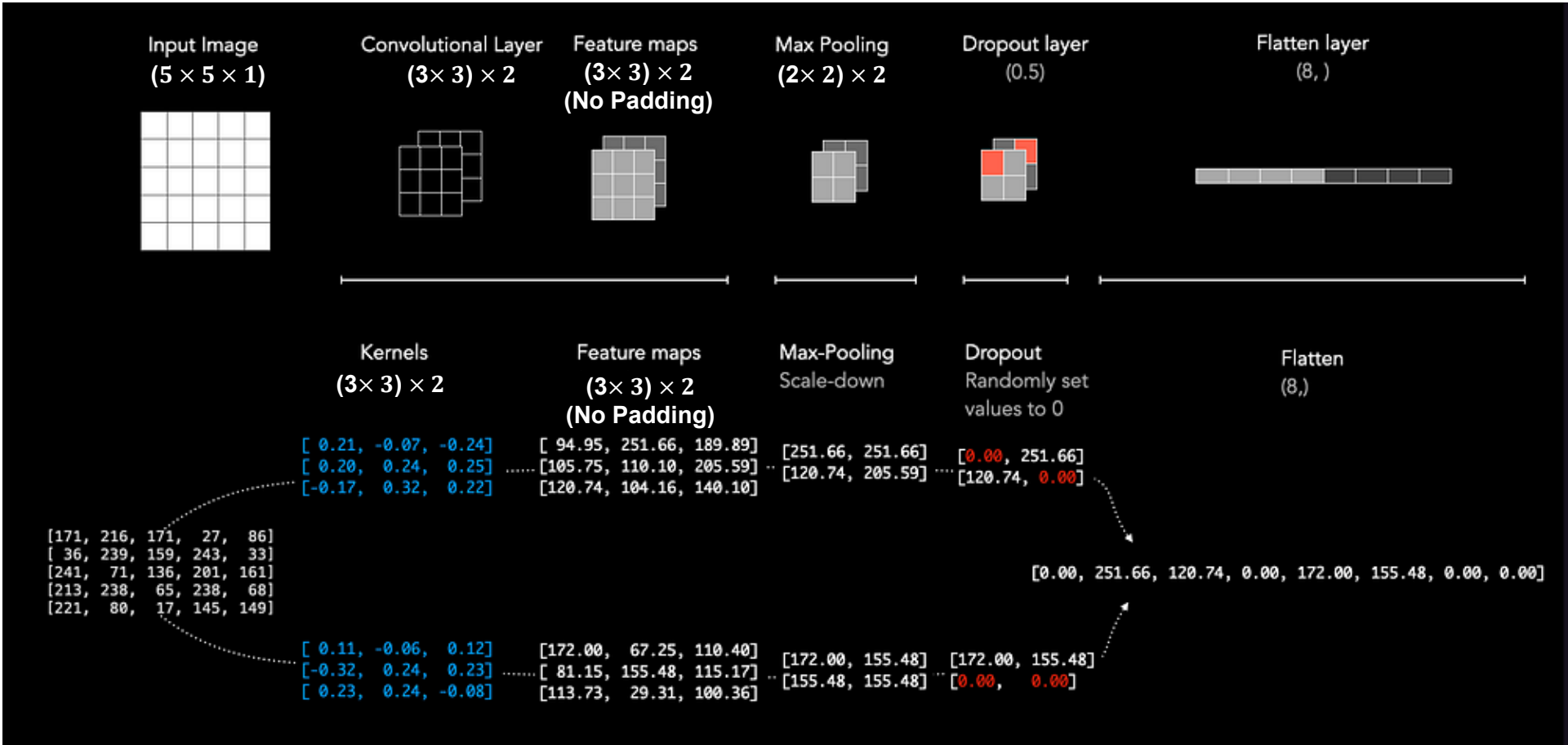
4×4 → 2×2 (2×2, stride 2)

- **Max pooling** keeps the strongest response;
- **Average pooling** keeps the mean
- Halves height and width, **no parameters**
- Adds small **shift invariance**, reduces computation
- Modern nets sometimes use stride-2 convs instead

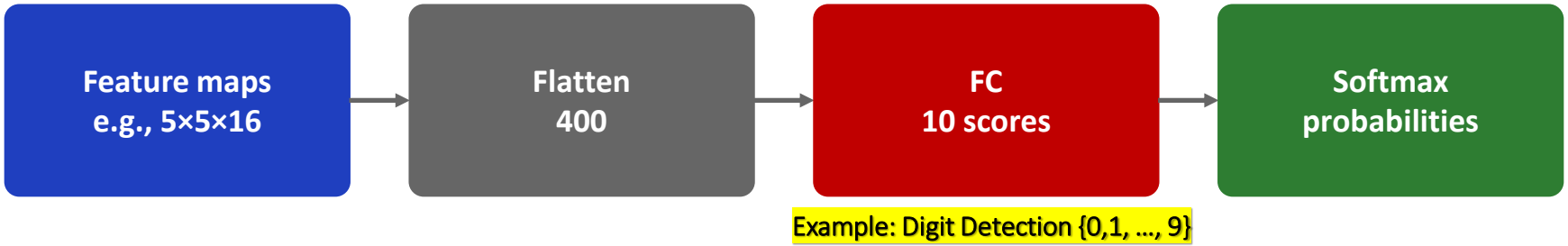
Pooling


max pooling
20 30
112 37
12 20 30 0
8 12 2 0
34 70 37 4
112 100 25 12
average pooling
13 8
79 20

Example: CNN pipeline on a given grayscale image

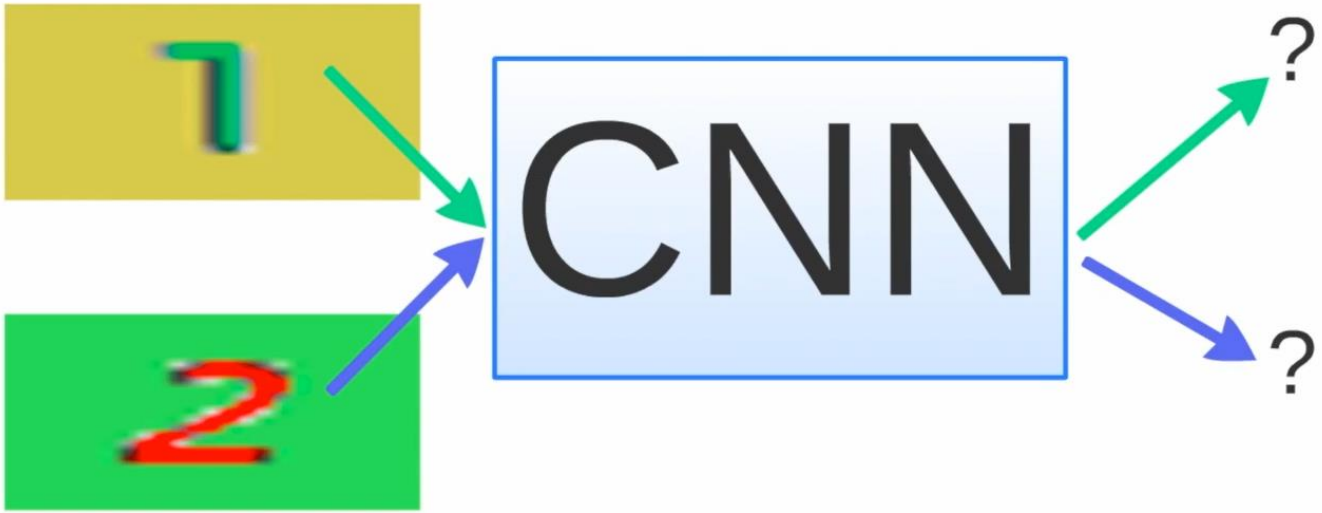


Flatten, Fully Connected, Softmax



- Flatten the final feature maps into a vector
- FC layers combine features into **class scores**
- Softmax: p_k = e^(z_k) / Σ e^(z_j); sums to 1
- Loss: **cross-entropy** (Lecture 2)
- Modern nets often replace big FC layers by **global average pooling**

Example: Digit Detection {0,1, …, 9}



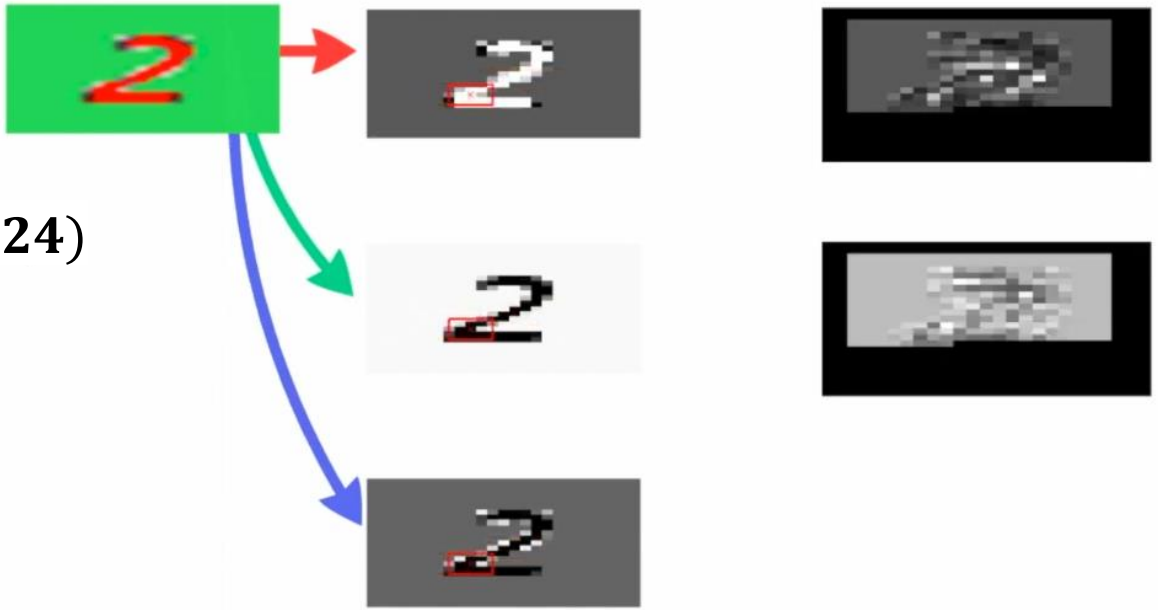
Apply Input Image 1 and we expect to have the output detecting digit 1
Similarly, Apply Input Image 2 and we expect to have the output detecting digit 2

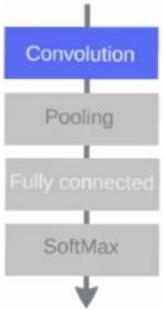
Example: Digit Detection {0,1, …, 9}: 1st Convolution

input:
- 3 channels image (24×24)

convolution:
- 2 filters 5x5
- ReLU function

output:
- 2 tensors (24×24)



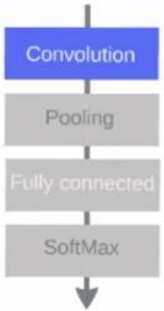


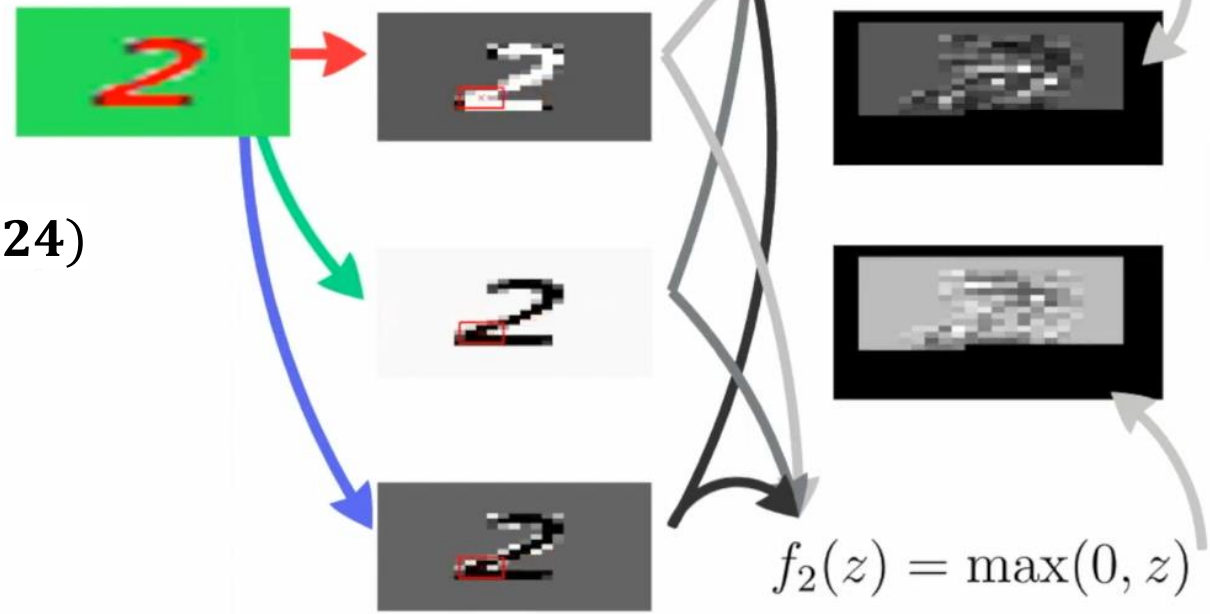
Example: Digit Detection {0,1, …, 9}: 1st Convolution

input:
- 3 channels image (24×24)

convolution:
- 2 filters 5x5
- ReLU function

output:
- 2 tensors (24×24)



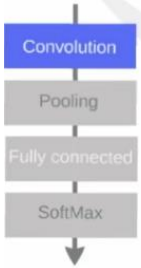


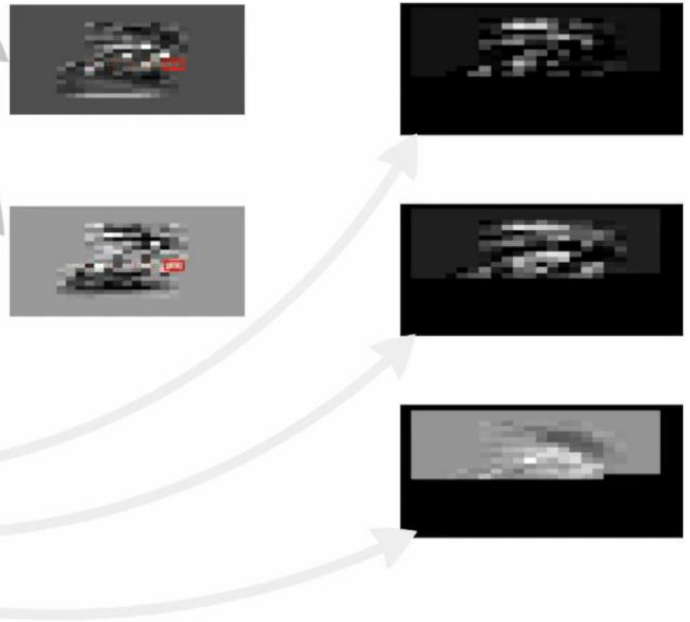
Example: Digit Detection {0,1, …, 9}: 2nd Convolution

input:
- 2 tensors from previous convolution (24×24)

convolution:
- 3 filters 3x3
- ReLU function

output:
- 3 tensors (24×24)





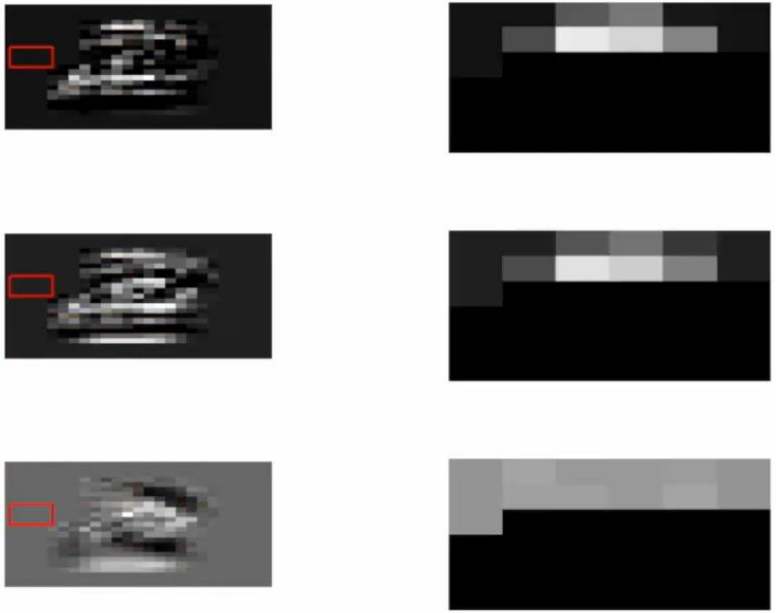
Example: Digit Detection {0,1, …, 9}: Pooling

input:
- 3 tensors from previous convolution (24×24)

pooling:
- 1 filter 4x4 (max function)

output:
- 3 tensors (6 x 6)



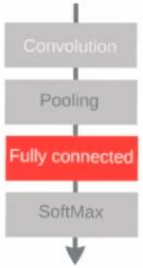


Example: Digit Detection {0,1, …, 9}: Fully Connected

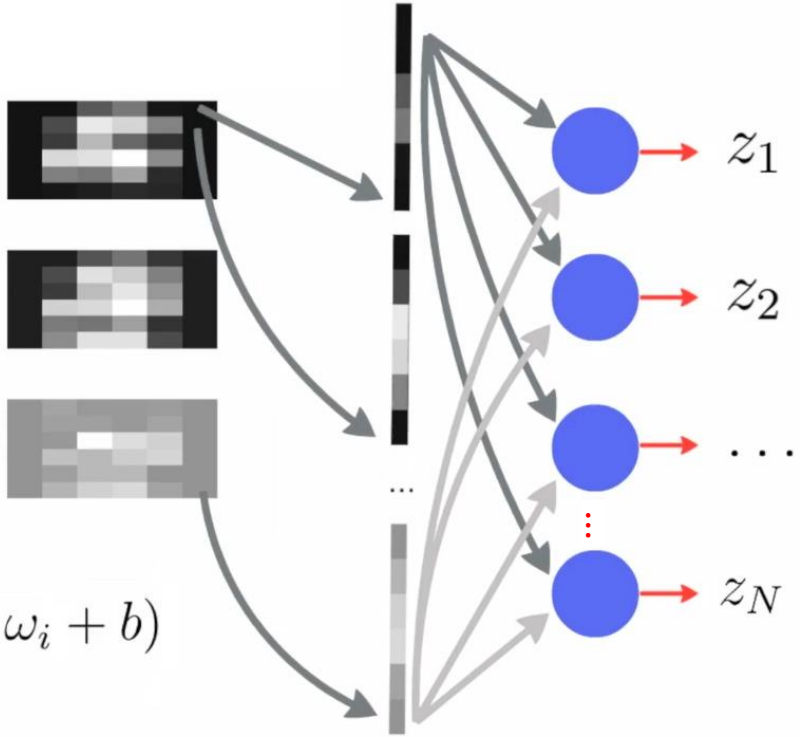
input:
- 3 tensors from previous convolution (6 x 6)

fully connected layer:
- 6x6x3 inputs

output:
- **10** neurons



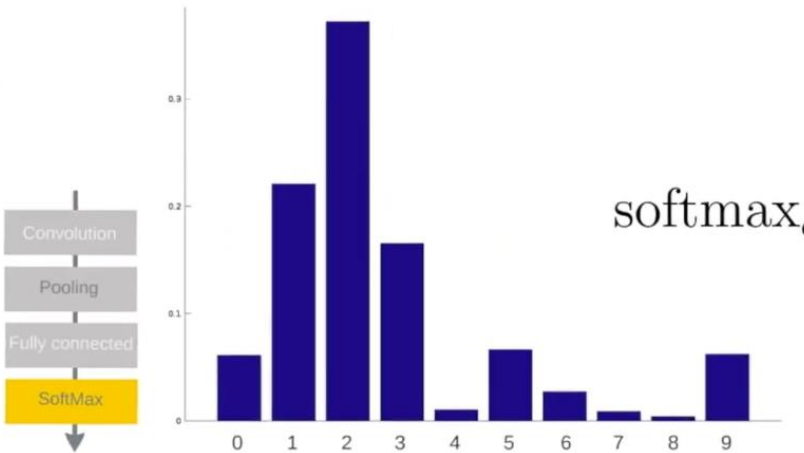
$$z = f(\sum_{i=1}^{N} x_i \times \omega_i + b)$$



Example: Digit Detection {0,1, …, 9}: Softmax

input:

- **10** neurons from previous layer



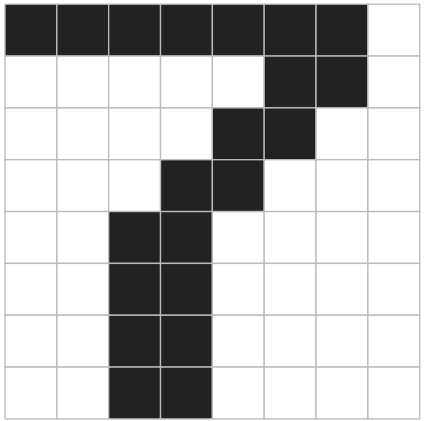
$$\text{softmax}_c(y = c) = \frac{e^{X \times \Omega_c + b_c}}{\sum_j e^{X \times \Omega_j + b_j}}$$

Output a value that relates the similarity of the input image to a single class (i.e. out of the 10 classes representing the digits 0, …, 9)

Digit Detection Example {0,…,9}: Tracing the Shapes

Layer	Output shape	Parameters
Input	24×24×3	0
Conv 5×5x3, 2 filters (padding=2) + ReLU	24×24×2	5×5×3×2+2 = 152
Conv 3×3, 3 filters (padding=1) + ReLU	24×24×3	3×3×2×3+3 = 57
Max pool 4×4, stride=4	6×6×3	0
Flatten	108	0
FC + softmax	10	108×10+10 = 1,090
Total		1,299

Recall FC Example (Lecture 1) : Recognizing Handwritten Digits



24×24 image (8×8 shown)



Parameters: 576×128+128 + 128×10+10 = **73,856 + 1290** = **75,146**

Output k = probability that the digit is k.

CNN vs Fully Connected on Digits

Fully connected

576 → 128 → 10
75,146 parameters
Flattens first: no spatial structure; breaks when digit shifts

CNN (above)

2 conv + 1 pool + FC
1,299 parameters
Keeps neighbourhoods; shared kernels; ~98–99% on MNIST

About 57× fewer parameters, and better accuracy.

Design Choices and Training

- Kernel size: **3×3** is the standard choice
- Number of filters usually **doubles** as size halves (32, 64, 128 …)
- Training is unchanged: loss → backprop → optimizer (Adam / SGD + momentum)
- Use **augmentation** and **dropout** against overfitting
- Next lecture: how these blocks were **stacked deeper**, from AlexNet to VGG

Recap

- **Convolution**: shared local kernels → feature maps
- **Stride, padding** set the output size: $O = (W-K+2P)/S + 1$
- **ReLU** adds non-linearity; **pooling** downsamples
- **Flatten + FC + softmax** classify the extracted features
- Next: **VGG, Inception** and what each stage adds